NAME: Reddy MOHINI

ROLL NO: A24126552110

BRANCH: CSM

SECTION: B

COURSE NAME: FULL STACK WEB DEVELOPMENT

CODE: 23CM4121

TITLE OF THE EXPERIMENT

Designing a Static Webpage Using HTML Components

AIM

To design and develop a **static webpage using HTML components** such as headings, paragraphs, images, hyperlinks, lists, tables, navigation menus, and forms

DESCRIPTION

This task focuses on creating a basic static webpage using **HTML (HyperText Markup Language)**. The webpage is designed using different HTML elements to organize and display content in a structured manner. It may include a **header, navigation menu, headings, text, images, hyperlinks, lists, tables, and a form**.

The main purpose is to understand the **structure of an HTML document** and learn how different HTML components are used to create a simple and user-friendly webpage. Since it is a static webpage, the content remains fixed and does not require a backend or database.

PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>

  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tr>
        <td>
```

```
            <h1>StaticWeb</h1>
          </td>
          <td align="right">
            <nav>
              <a href="#home">Home</a>  | 
              <a href="#articles">Articles</a>  | 
              <a href="#about">About</a>  | 
              <a href="#contact">Contact Us</a>
            </nav>
          </td>
        </tr>
      </table>
    </header>

    <hr>

    <!-- Hero / Showcase Section -->
    <section id="home">
      <center>
        <h2>Semantic Structures Without CSS</h2>
        <p>This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.</p>
        <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
      </center>
    </section>

    <hr>

    <!-- Main Content Layout using a Structural Table -->
    <table width="100%" border="0" cellspacing="0" cellpadding="20">
      <tr valign="top">

        <!-- Left Side: Main Content Column -->
        <td width="70%">
          <main id="articles">

            <article>
              <h2>1. Core Structural Tags</h2>
              <p>Published: July 7, 2026</p>
              <p>When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and <code>&lt;section&gt;</code> tell screen readers and web scrapers exactly how information flows.</p>
              <p>Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.</p>
            </article>
```

```
            <br><hr><br>

            <article>
              <h2>2. Native Interaction Elements</h2>
              <p>Published: July 5, 2026</p>
              <p>Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.</p>

              <h3>Interactive Demonstration Forms</h3>
              <form id="contact" action="#" method="post">
                <fieldset>
                  <legend><b>User Inquiry Form</b></legend>
                  <p>
                    <label for="name">Full Name: </label>
                    <input type="text" id="name" name="username" required placeholder="John Doe">
                  </p>
                  <p>
                    <label for="email">Email Address: </label>
                    <input type="email" id="email" name="useremail" required>
                  </p>
                  <p>
                    <label for="topic">Inquiry Topic: </label>
                    <select id="topic" name="usertopic">
                      <option value="general">General Feedback</option>
                      <option value="technical">Technical Architecture</option>
                      <option value="academics">Academic Curriculum</option>
                    </select>
                  </p>
                  <p>
                    <label for="message">Message Details:</label><br>
                    <textarea id="message" name="usermessage" rows="5" cols="40" placeholder="Type your notes here..."></textarea>
                  </p>
                  <p>
                    <input type="submit" value="Submit Form Data">
                    <input type="reset" value="Clear Entries">
                  </p>
                </fieldset>
              </form>
            </article>

          </main>
        </td>
```

```
        <!-- Right Side: Sidebar Column -->
        <td width="30%" bgcolor="#f4f4f4">
          <aside id="about">
            <h3>Document References</h3>
            <p>Static pages map data out directly without real-time database queries or
processor overhead.</p>

            <h4>Core Benefits:</h4>
            <ul>
              <li>Instant browser painting</li>
              <li>Low memory footprints</li>
              <li>High data compatibility</li>
            </ul>

            <h4>Required Components:</h4>
            <ol>
              <li>Document Type Definition</li>
              <li>Semantic Wrapper Blocks</li>
              <li>Data Input Control Interfaces</li>
            </ol>
          </aside>
        </td>

      </tr>
    </table>

    <hr>

    <!-- Footer Section -->
    <footer>
      <center>
        <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
      </center>
    </footer>

</body>
</html>
```

output

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

Read Our Articles Below ↓

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

---

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

---

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name: John Doe

Email Address:

Inquiry Topic: General Feedback

Message Details:

Type your notes here...

Submit Form Data | Clear Entries

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces